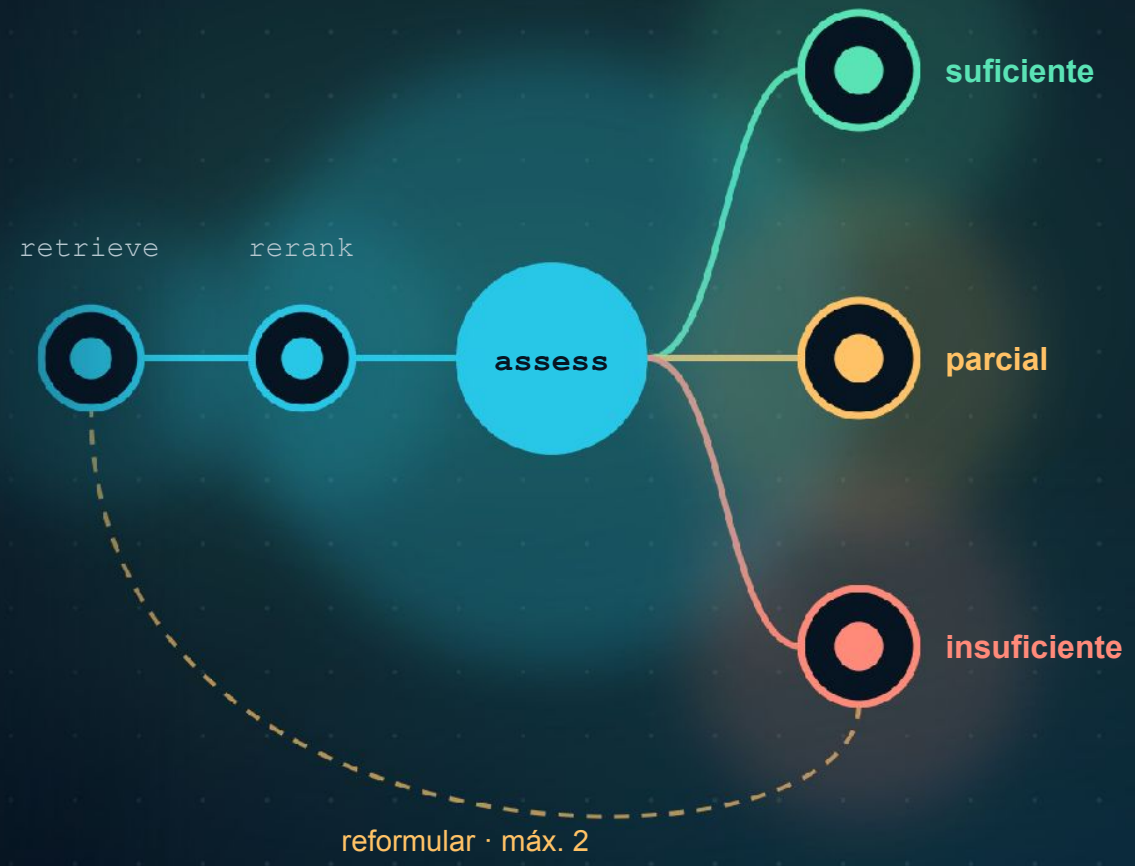
thePower

S11 · AI ENGINEER

CRAG

RAG avanzado: cuando una búsqueda no basta

Reordenar, evaluar la evidencia y corregir la recuperación, con límites y medido frente al RAG básico.



José Antonio Nuevo

Martes 27 de octubre de 2026 · 18:30–20:00

ANTES DE EMPEZAR · 1 MINUTO

Evalúa la clase anterior: S10 · RAG



Encuesta AI Engineer

Escanea el QR para evaluar la clase anterior

RAG EN UN MARKETPLACE

Descripciones y datos actuales

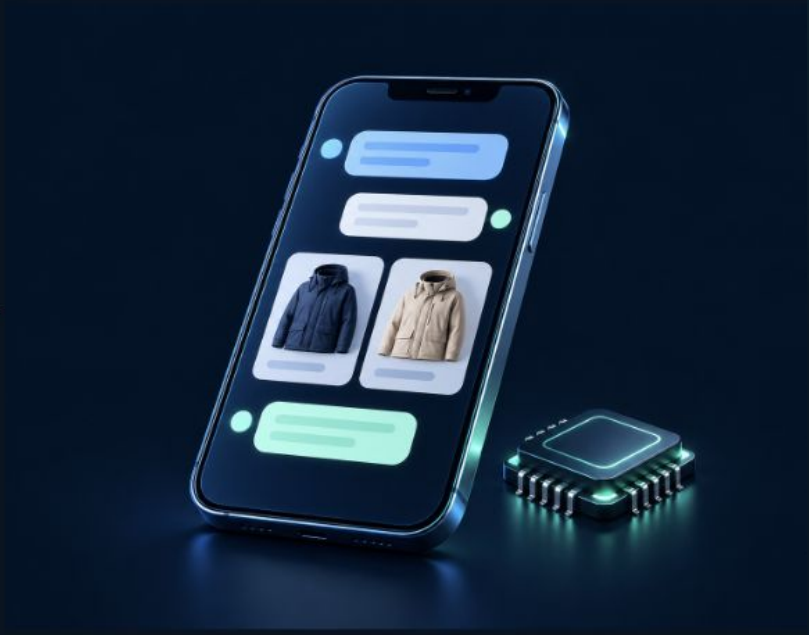
La recomendación necesita evidencia sobre la prenda y condiciones comerciales vigentes.

Descripción de la prenda

Uso, estilo, materiales y protección frente a la lluvia. Buscamos el pasaje que respalda cada preferencia.

Catálogo comercial actual

Precio, talla y stock se consultan como datos estructurados. La aplicación comprueba qué se puede comprar ahora.



Aplicación + LLM

API o MCP pueden exponer las operaciones de ambas fuentes.

Que es una base de datos vectorial?

Una base de datos vectorial es un sistema diseñado para almacenar y buscar información mediante vectores numéricos que representan características o significados.



Tecnología	Tipo	Principal ventaja	Principal inconveniente
pgvector	Extensión PostgreSQL	SQL, JOIN y vectores en la misma BD	Requiere optimización al escalar
Qdrant	Especializada	Buen filtrado, búsqueda híbrida y control	Infraestructura adicional
Pinecone	Especializada cloud	Servicio gestionado y escalabilidad	Dependencia del proveedor
Weaviate	Especializada	Búsqueda híbrida y módulos de IA	Más componentes que configurar
Milvus	Especializada	Grandes volúmenes y despliegue distribuido	Mayor complejidad operativa
Chroma	Especializada	Desarrollo local y prototipos rápidos	Hay que evaluar su despliegue y escala
Cloudflare Vectorize	Cloud serverless	Integración con Workers	Límites específicos del servicio
Elasticsearch	Motor de búsqueda	Texto completo, vectores y filtros	Operación y recursos
Redis	Base en memoria	Búsqueda rápida y caché semántica	Costes de memoria e infraestructura
MongoDB Atlas Vector Search	Documental	Vectores y documentos en MongoDB	Dependencia de sus índices y plataforma
PlanetScale	SQL con vectores	Búsqueda vectorial dentro de MySQL	Evaluar compatibilidad y límites concretos

pgvector vs Qdrant vs Pinecone: cuál elegir?



1. pgvector

Una base PostgreSQL que también sabe buscar vectores.

Lo elegiría si ya utilizas PostgreSQL y quieres mantener los datos de negocio y los embeddings en un único sistema.



2. Qdrant

Un motor diseñado específicamente para búsquedas vectoriales.

Lo elegiría si el sistema RAG o la búsqueda semántica tienen mucho peso en el producto y necesito un control detallado del retrieval.



3. Pinecone

Búsqueda especializada como servicio cloud.

Lo elegiría si quiero una plataforma gestionada, escalado automático y menos responsabilidad operativa.

Característica	pgvector	Qdrant	Pinecone
Qué es	Extensión de PostgreSQL	Base vectorial independiente	Servicio de búsqueda gestionado
Base de datos relacional completa	Sí, PostgreSQL	No	No
SQL y JOIN	Nativos	No como PostgreSQL	No como PostgreSQL
Búsqueda vectorial	Sí	Sí	Sí
Búsqueda híbrida	Combinando funciones de PostgreSQL	Integrada	Integrada
Filtros por metadatos	SQL	Payload filters	Metadata filters
HNSW configurable	Sí	Sí	Gestionado internamente
Self-hosting	Sí	Sí	No en modalidad OSS
Servicio cloud gestionado	Mediante proveedores PostgreSQL	Qdrant Cloud	Pinecone
Escalado distribuido	Requiere arquitectura adicional	Soportado	Gestionado por el servicio
Lock-in de proveedor	Bajo	Bajo en modalidad OSS	Mayor
Uso principal	Aplicaciones SQL + IA	Retrieval especializado	Retrieval gestionado

pgvector: extensión de PostgreSQL que permite almacenar y buscar vectores mediante SQL, sin necesitar otra base de datos.
Qdrant: base de datos especializada en búsquedas vectoriales rápidas, con filtrado avanzado y búsqueda híbrida.
Pinecone: servicio cloud gestionado de búsqueda vectorial que facilita el escalado sin administrar infraestructura.

Cómo se obtiene un embedding

Un modelo convierte el texto en un vector útil para comparar contenidos.



Un embedding representa el contenido en un espacio numérico compartido.

Texto	«Membrana impermeable y costuras selladas»
Tokenizador	Divide el texto en unidades del modelo.
Encoder	Procesa las relaciones entre esas unidades.
Embedding	[0,09; −0,18; 0,85; 0,12]

Fichas y preguntas usan un espacio compatible. Vector sintético de 4 dimensiones. Un token puede ser parte de una palabra.

FUNDAMENTOS

Qué guarda una base vectorial

Conserva la relación entre cada vector, el texto original y su procedencia.



Índice de búsqueda

Acelera la localización de vecinos. La búsqueda puede ser exacta o aproximada.

Cada fila corresponde a un chunk

Chunk	Texto	Vector
P101-01	Uso y estilo	[0,22; −0,31; …]
P101-02	Lluvia	[0,09; −0,18; …]
P101-03	Ajuste	[0,30; −0,21; …]



ID + texto → Producto + versión → pgvector + SQL

Vectores sintéticos. Metadatos: P101 y v1.

RECORRIDO GENERAL

Las dos fases de RAG

RAG recupera información externa, la añade al contexto y genera una respuesta apoyada en ella.

INGESTA

Cuando cambia la ficha



Documento
Ficha de Nube

Chunks
P101-01, 02 y 03

Embeddings
Un vector por chunk

Base e índice
Texto, vector y origen

CONSULTA

Cada pregunta del comprador



Pregunta
Intención y filtros

Embedding
Vector de consulta

Recuperación
Fichas y catálogo

Contexto
Texto seleccionado

LLM
Respuesta con fuentes

DOCUMENTO E INGESTA

La ficha de Nube antes de dividirla

Extraemos el texto del catálogo y conservamos producto, versión y estructura.



P101

Chaqueta Nube
Ficha v1

Texto completo
186 palabras

Nube es una chaqueta urbana ligera pensada para desplazamientos cotidianos y jornadas de trabajo. Su diseño liso, de líneas rectas y sin logotipos grandes combina con pantalones de vestir, vaqueros o ropa informal. La silueta evita detalles deportivos de montaña. Su tejido ocupa poco espacio y permite guardarla en una mochila. Es una prenda discreta para moverse por la ciudad.

La protección frente a la lluvia procede de una membrana impermeable y de las costuras selladas. La capucha ajustable cubre la cabeza y una solapa interior protege la cremallera. Estas características permiten usar Nube bajo lluvia durante los desplazamientos urbanos. La impermeabilidad es distinta de un tratamiento que solo repele gotas superficiales. La prenda no es equipamiento especializado para actividades prolongadas en alta montaña.

El corte regular deja espacio para una camisa o una capa fina debajo sin una apariencia voluminosa. Los puños se ajustan para reducir la entrada de agua y los bolsillos laterales llevan cierre. La capucha puede recogerse para conseguir un aspecto más sobrio. Para conservar la prenda, sigue las instrucciones de cuidado de la etiqueta y evita tratamientos que deterioren la membrana.

Ficha ficticia. Precio y stock viven en el catálogo comercial.

DOCUMENTO E INGESTA

Tres párrafos, tres chunks

Cada chunk conserva una unidad de texto y recibe un ID estable. El contenido sigue vinculado a P101.



P101-01

Uso y estilo

Nube es una chaqueta urbana ligera pensada para desplazamientos cotidianos y jornadas de trabajo. Su diseño liso, de líneas rectas y sin logotipos grandes combina con pantalones de vestir, vaqueros o ropa informal. La silueta evita detalles deportivos de montaña. Su tejido ocupa poco espacio y permite guardarla en una mochila. Es una prenda discreta para moverse por la ciudad.



P101-02

Protección de lluvia

La protección frente a la lluvia procede de una membrana impermeable y de las costuras selladas. La capucha ajustable cubre la cabeza y una solapa interior protege la cremallera. Estas características permiten usar Nube bajo lluvia durante los desplazamientos urbanos. La impermeabilidad es distinta de un tratamiento que solo repele gotas superficiales. La prenda no es equipamiento especializado para actividades prolongadas en alta montaña.



P101-03

Ajuste y detalles

El corte regular deja espacio para una camisa o una capa fina debajo sin una apariencia voluminosa. Los puños se ajustan para reducir la entrada de agua y los bolsillos laterales llevan cierre. La capucha puede recogerse para conseguir un aspecto más sobrio. Para conservar la prenda, sigue las instrucciones de cuidado de la etiqueta y evita tratamientos que deterioren la membrana.

Cabecera contextual + texto original para el embedding. Cada ID conserva producto, versión y origen.

Límites de tokens y solapamiento

Los párrafos largos se dividen en unidades menores. El overlap repite una parte en el siguiente chunk.



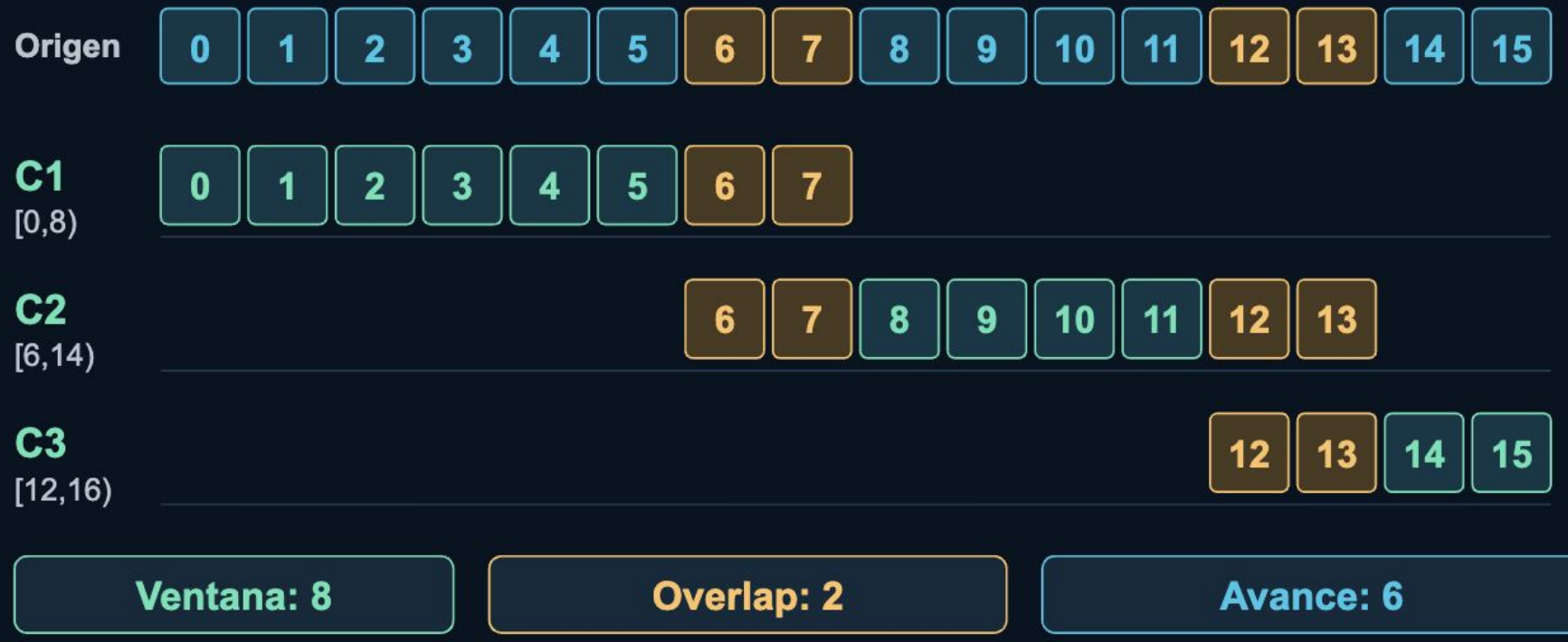
REGLA DE CORTE

Párrafos y frases primero.

Si el bloque no cabe, usamos ventanas.

La cabecera también ocupa parte del límite.

Ejemplo didáctico: 16 posiciones de tokens



16 tokens únicos y 20 procesados contando repeticiones. La ficha anterior usa 3 párrafos completos sin overlap.

Embedding e inserción del chunk

La aplicación calcula el vector y guarda un registro que permite recuperar el mismo texto.

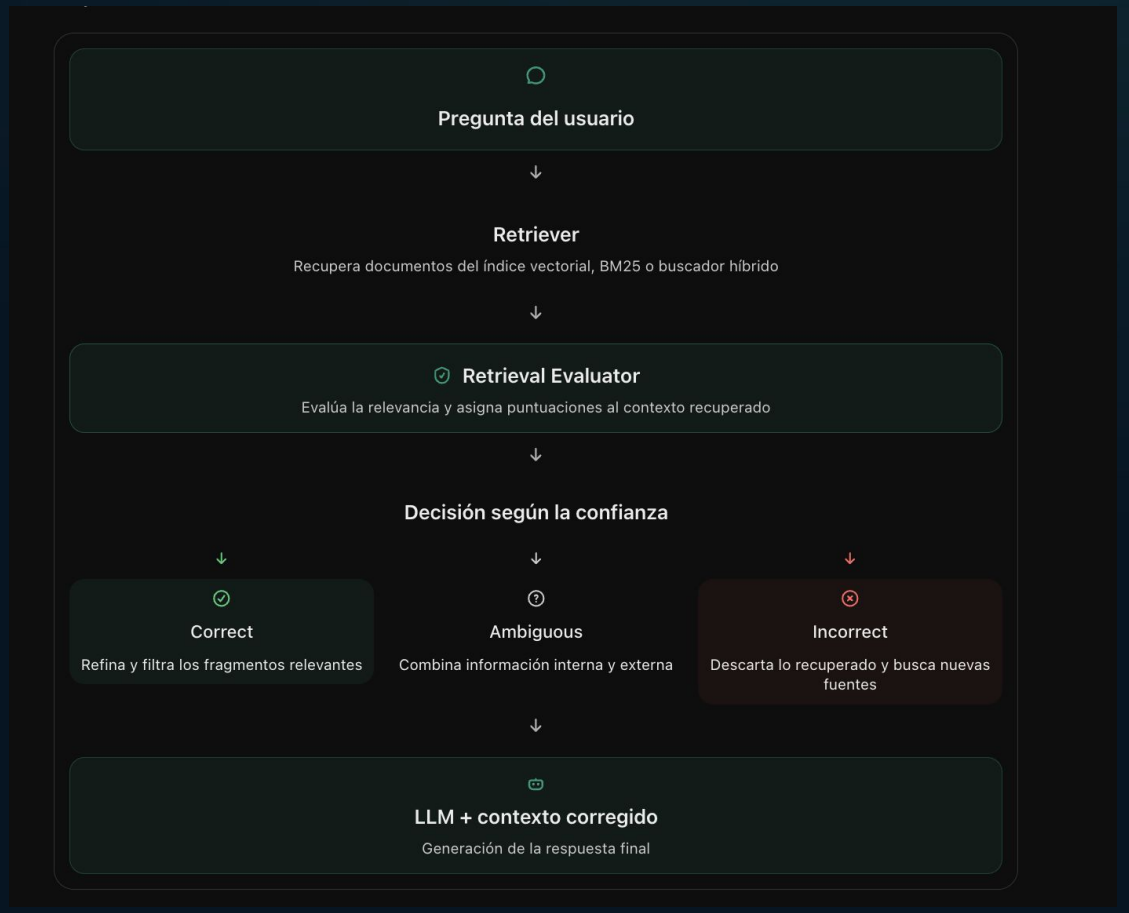


Si cambia la descripción, recalculamos el vector. Precio y stock se mantienen en sus tablas.

RAG


1. Documentos originales
2. Chunks + embeddings
3. Índice de búsqueda
Pregunta usuario
4. Recuperar Top-K chunks
5. Prompt + contexto → LLM
Respuesta generada

CRAG


Pregunta del usuario
Retriever
Recupera documentos del índice vectorial, BM25 o buscador híbrido
Retrieval Evaluator
Evalúa la relevancia y asigna puntuaciones al contexto recuperado
Decisión según la confianza
Correct
Refina y filtra los fragmentos relevantes
Ambiguous
Combina información interna y externa
Incorrect
Descarta lo recuperado y busca nuevas fuentes
LLM + contexto corregido
Generación de la respuesta final

Como se comporta el CRAG según el caso

Caso 1 — Correct: la información recuperada parece relevante

El evaluador considera que existe suficiente evidencia relevante.

El sistema no necesita volver a buscar desde cero. En su lugar, refina los documentos recuperados, identifica las secciones útiles y elimina ruido.

Caso 2 — Incorrect: la recuperación ha fallado

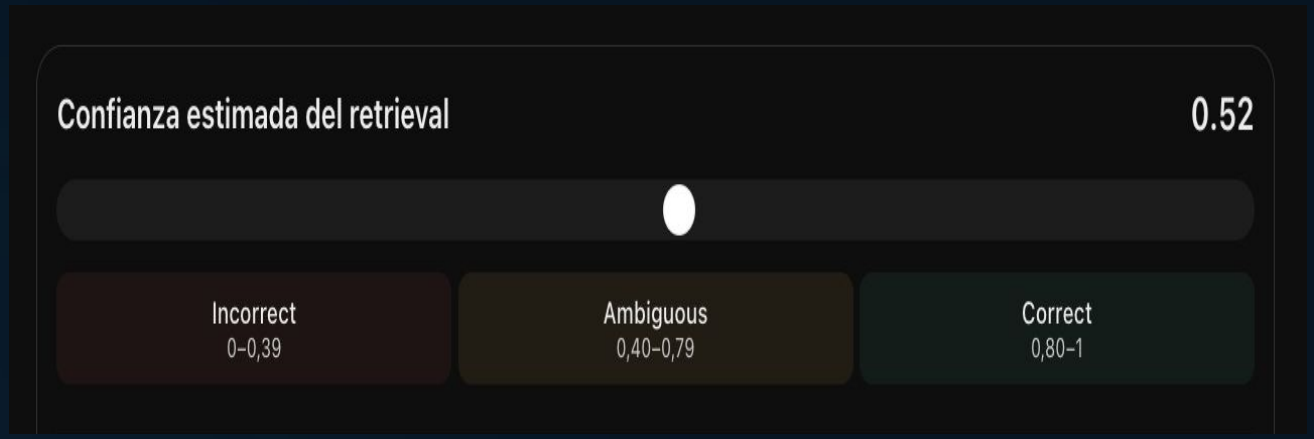
Ninguno de los documentos recuperados parece suficientemente relevante.

En la propuesta original, se descartan esos resultados, se reformula la pregunta como consulta de búsqueda y se buscan nuevas fuentes mediante un buscador web.

Caso 3 — Ambiguous: la recuperación es incierta

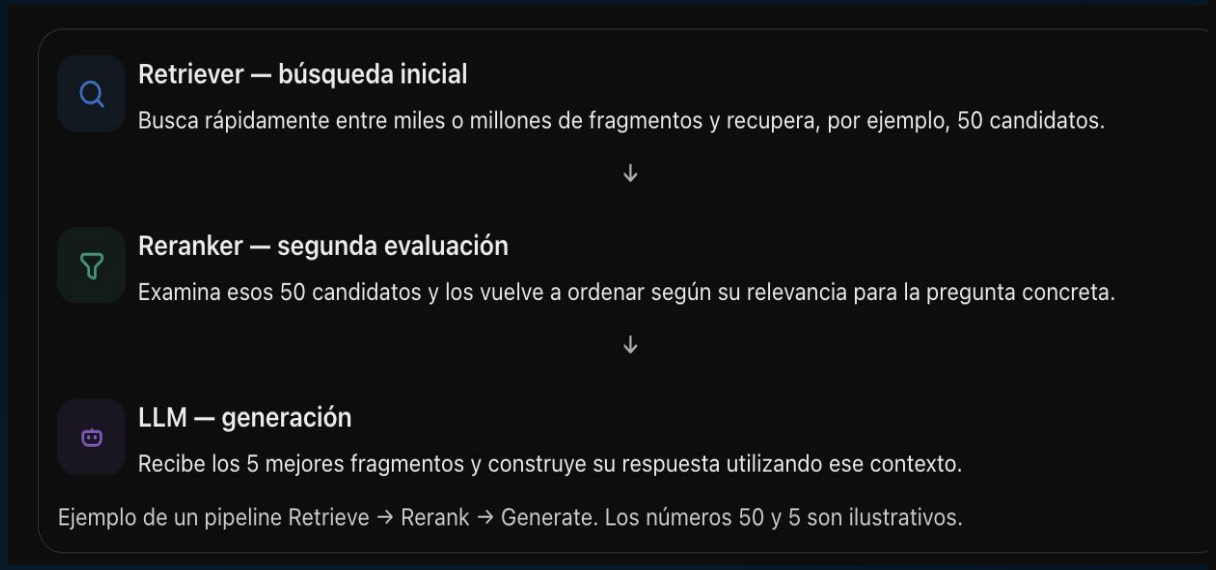
La evidencia encontrada puede resultar útil, pero el evaluador no tiene suficiente confianza.

CRAG combina los fragmentos internos refinados con información adicional encontrada en fuentes externas.



Reranking

El retriever encuentra documentos que podrían ser relevantes. El reranker analiza esos documentos con mayor profundidad y decide cuáles responden mejor a la pregunta del usuario.



Ejemplo de un pipeline Retrieve → Rerank → Generate. Los números 50 y 5 son ilustrativos.

Retriever vs reranker

	Retriever	Reranker
Objetivo	Encontrar candidatos	Ordenar los candidatos
Prioridad	Velocidad y cobertura	Precisión del ranking
Volumen habitual	Miles o millones de documentos indexados	Decenas o cientos de candidatos
Modelo típico	Bi-encoder / BM25	Cross-encoder
Procesamiento	Búsqueda por vectores o términos	Evaluación de cada pareja pregunta-documento
Resultado	Top-K candidatos	Candidatos reordenados
¿Encuentra documentos nuevos?	Sí, en el corpus	No, reordena los recibidos

¿Dónde se ejecuta exactamente el reranking?

El reranking se ejecuta normalmente en el backend, durante el procesamiento de cada consulta, después del retrieval y antes de llamar al LLM.

Proceso A: indexación de documentos

Ocurre cuando añadimos o actualizamos documentos.

Indexación — antes de las preguntas

- Documentos
- Chunking
- Embeddings
- Índice vectorial

Aquí no solemos ejecutar un reranker específico para la pregunta, porque todavía no conocemos la pregunta.

Proceso B: ejecución de una pregunta

Se inicia cada vez que un usuario consulta al asistente.

1. **Pregunta del usuario**
   ¿Cómo funciona OAuth en API v2?
2. **Embedding de la pregunta**
   Vector numérico para la búsqueda semántica
3. **Retrieval**
   PostgreSQL / Qdrant / Elasticsearch → 50 candidatos
4. **RERANKING** ←
   Evalúa los 50 candidatos → selecciona los 5 mejores
5. **Construcción del contexto**
   Combina fragmentos, metadatos y fuentes
6. **LLM**
   Genera la respuesta a partir de la evidencia

EL LÍMITE DEL RAG BÁSICO

Una respuesta puede sonar bien y estar incompleta



El fallo no está en el generador: está en aceptar evidencia parcial como si bastara.

RAG AVANZADO

Dónde se puede mejorar un RAG básico

Objetivo del temario: superar sus limitaciones con autocorrección y reordenamiento de resultados.



Lo que usa el CRAG de S11: reformulación, top-10, reordenamiento, evaluación de la evidencia y rechazo, en un mismo grafo.

CORRECTIVE RAG

CRAG mete un juicio entre recuperar y generar

Recuperar

`retrieve`

Trae hasta 10 candidatos desde el índice vectorial.

→

Reordenar

`rerank`

Puntúa cada fragmento en [0, 1] frente a la consulta.

→

Juzgar

`assess`

¿La evidencia útil cubre toda la pregunta?

→

Corregir

`retain · discard`

Conserva o descarta, reformula y vuelve a buscar. Si basta, responde.

✓ **Qué es** Una política de recuperación observable: cada decisión e intento queda en la traza.

✗ **Qué no es** No es GraphRAG, no entrena el modelo y no garantiza mejorar cada respuesta.

Inspirado en Yan, Gu, Zhu y Ling (2024), Corrective Retrieval Augmented Generation, arXiv:2401.15884. Es una adaptación docente, no una reproducción del artículo.

PUNTO DE PARTIDA

Mismo asistente, otra política de recuperación

Pieza	RAG básico · S10	CRAG · S11
Búsquedas	1 búsqueda (top-4)	**Hasta 3 (top-10 y 2 reformulaciones)**
Control	Un umbral de similitud	**rerank en [0, 1] + assess de cobertura**
Contexto	Lo recuperado una vez	**Conserva, une sin duplicados o descarta**
Decisión	Responder o rechazar	**Suficiente, parcial o insuficiente; responde o rechaza**
Coste	Una recuperación	**Más llamadas y más latencia**

IGUAL EN LOS DOS

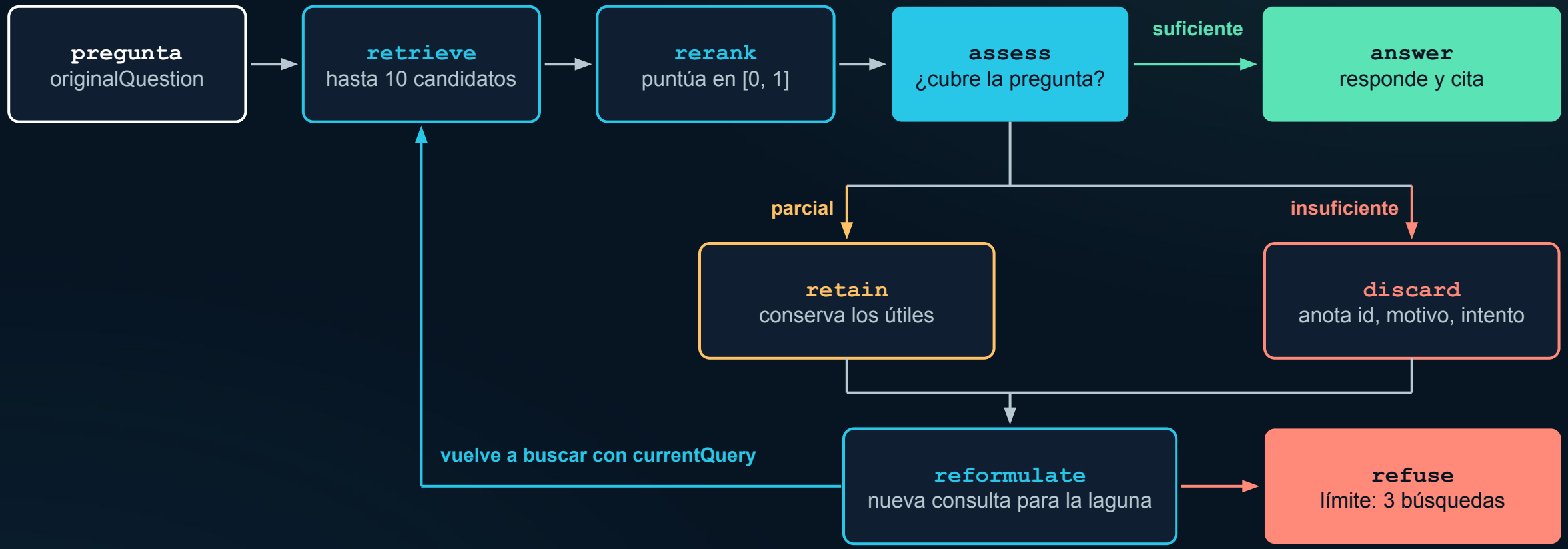
- Corpus y fragmentos
- Índice y modelo de embeddings
- Modelo generativo
- Instrucciones base
- Conversaciones limpias

Solo cambia la política de corrección.

El RAG básico sigue en el mismo proyecto: es la línea base contra la que medimos CRAG.

ARQUITECTURA

El grafo de LangGraph, nodo a nodo



Cada nodo devuelve datos estructurados; las aristas condicionales deciden la ruta. refuse termina sin llamar al generador.

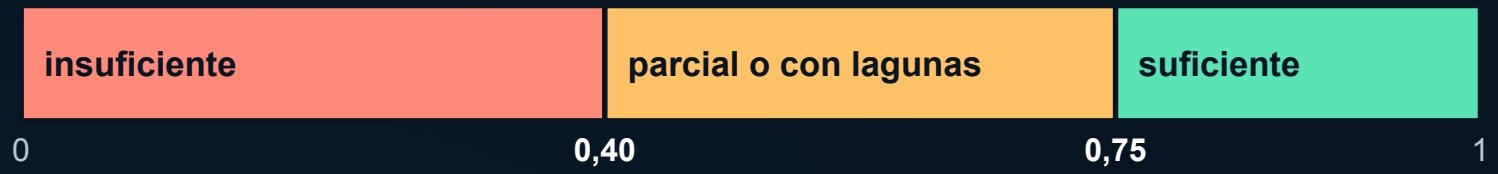
REORDENAR Y EVALUAR

Relevancia de un fragmento ≠ cobertura de la pregunta

rerank ¿Este fragmento es relevante para la consulta?



assess ¿La evidencia útil cubre toda la pregunta?



EJEMPLO DE LA DEMO

Fragmento de S5

`rerank = 0,80 → útil`

Pero falta la observabilidad

`cobertura = 0,50 → parcial`

Un fragmento muy relevante puede cubrir solo media pregunta.

Umbrales calibrados con ejemplos de validación, nunca con el test. Una puntuación no es una probabilidad de acierto.

LAS RUTAS CORRECTIVAS

Parcial conserva; insuficiente descarta

PARCIAL *guion «Evidencia parcial y segunda búsqueda»*

«¿Cómo modela LangGraph el estado de un agente y cómo se observa esa ejecución?»

1. `retrieve 1 · assess`
   S5 (0,80) es útil, pero la cobertura es 0,50
2. `retain`
   Conserva S5 con su procedencia
3. `reformulate`
   «observabilidad Langfuse y LangSmith de un agente»
4. `retrieve 2 · answer`
   S6 (0,86) + S5 sin duplicar: responde citando ambos

INSUFICIENTE *guion «Primera evidencia descartada»*

«¿Qué cubre la sesión de observabilidad del programa?»

1. `retrieve 1 · assess`
   S16 · Cloud AWS (0,20 < 0,50): nada útil
2. `discard`
   Anota id, motivo e intento 1
3. `reformulate`
   Nueva consulta desde la pregunta original
4. `retrieve 2 · answer`
   S6 (0,88): responde. S16 nunca entra en el prompt

ESTADO DEL TURNO

La pregunta original nunca se sobrescribe

`originalQuestion`

Lo que preguntó el alumno. Llega intacta a assess y a answer.

`currentQuery`

La consulta de búsqueda. Cambia en cada reformulación.

`queries`

Consultas ya usadas: una consulta repetida corta el bucle.

Cada turno nace vacío: contadores a 0 y sin checkpointer que arrastre estado de otra pregunta.

lab/src/crag-graph.ts · extracto

```
Annotation.Root({
  originalQuestion: Annotation<string>,
  currentQuery: Annotation<string>,
  queries: Annotation<string[]>,
  retrievalCount: Annotation<number>,      // máx. 3
  reformulationCount: Annotation<number>, // máx. 2
  retained: Annotation<LabVectorHit[]>,
  discarded: Annotation<LabDiscardedRecord[]>,
  terminal: Annotation<"pending" | "answer" | "refuse">,
})
```

EVALUACIÓN Y MÉTRICAS

Qué medir en un sistema RAG

RECUPERACIÓN

¿Llega la evidencia?

- Recall@k: ¿está el fragmento esperado en el top-k?
- MRR y nDCG: ¿en qué posición aparece?
- Precisión del contexto: ¿cuánto ruido entra?

GENERACIÓN

¿La respuesta se apoya en ella?

- **Acierto frente a la referencia (X/14)**
- **Fuentes citadas revisadas a mano**
- Fidelidad: nada fuera del contexto

ABSTENCIÓN

¿Sabe decir «no consta»?

- **Rechazos correctos (Y/6)**
- Falsos rechazos en preguntas con respuesta
- Datos inventados sin evidencia

OPERACIÓN

¿Cuánto cuesta corregir?

- **Búsquedas y reformulaciones medias**
- **Latencia mediana y media**
- Tokens y coste conocido

- Pedido en el ejercicio S11. Ragas automatiza fidelidad y precisión del contexto con un modelo juez; aquí no se exige.

1 · Desarrollo
casos visibles

→

2 · Validación
calibrar umbrales y prompts

→

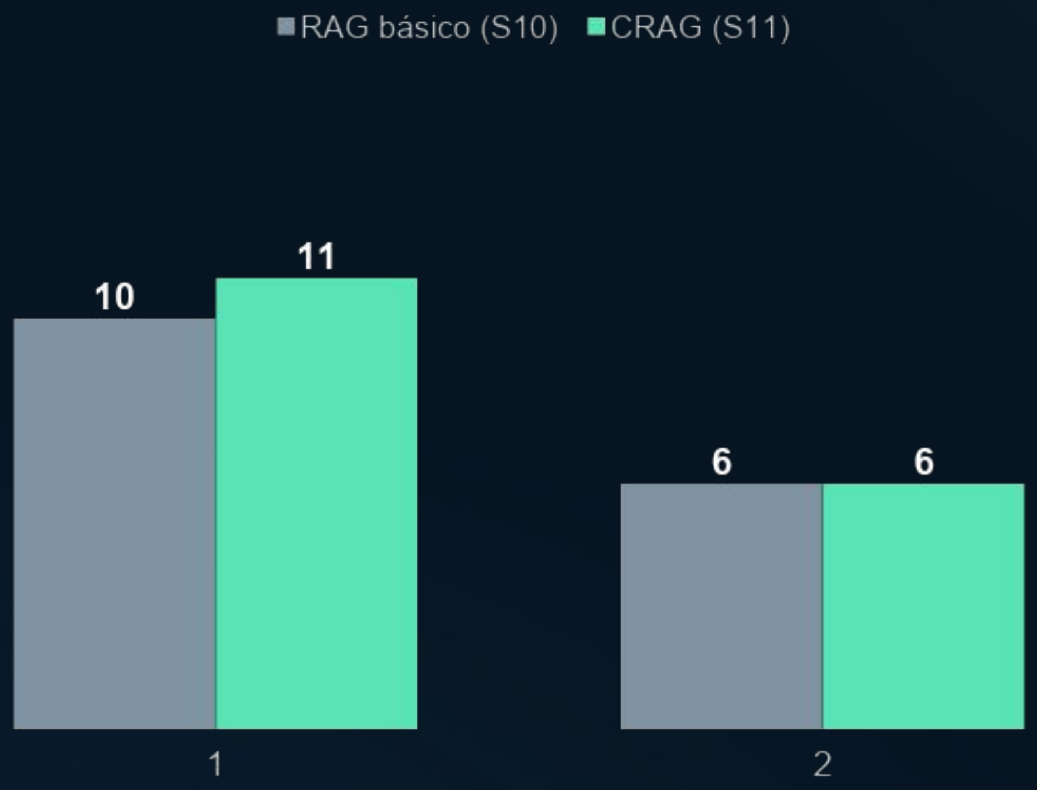
3 · Congelar
configuración y versiones

→

4 · Test
20 nuevas: 14 + 6

RESULTADOS DEL LABORATORIO · 20 PREGUNTAS

CRAG acierta una más y casi duplica las búsquedas



16 → 17

Aciertos sobre 20. CRAG gana las preguntas que unen dos documentos (casos 5, 10 y 14).

1,00 → 1,80

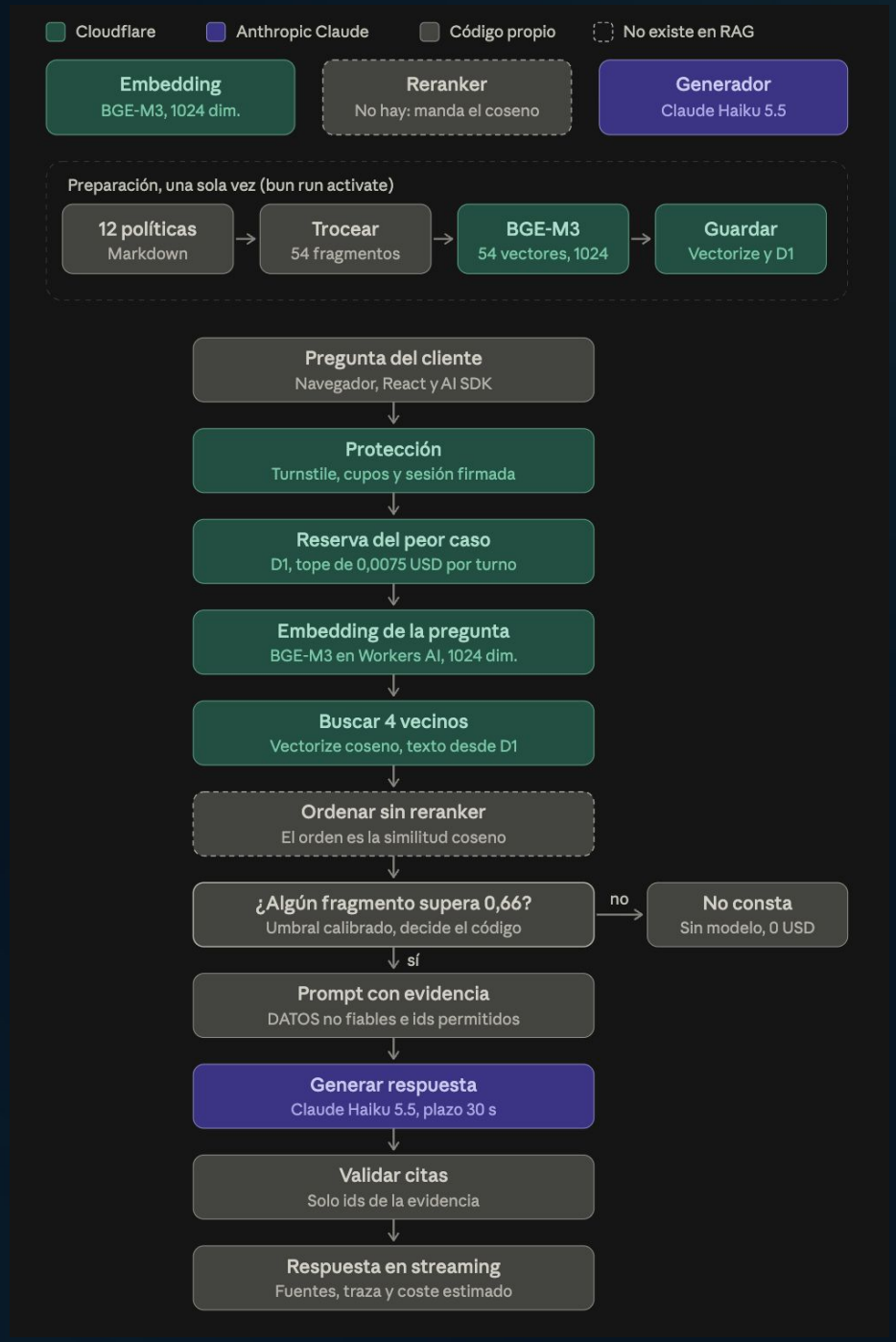
Búsquedas medias. Cada pregunta sin respuesta agota 3 búsquedas antes de rechazar.

2 fallos nuevos

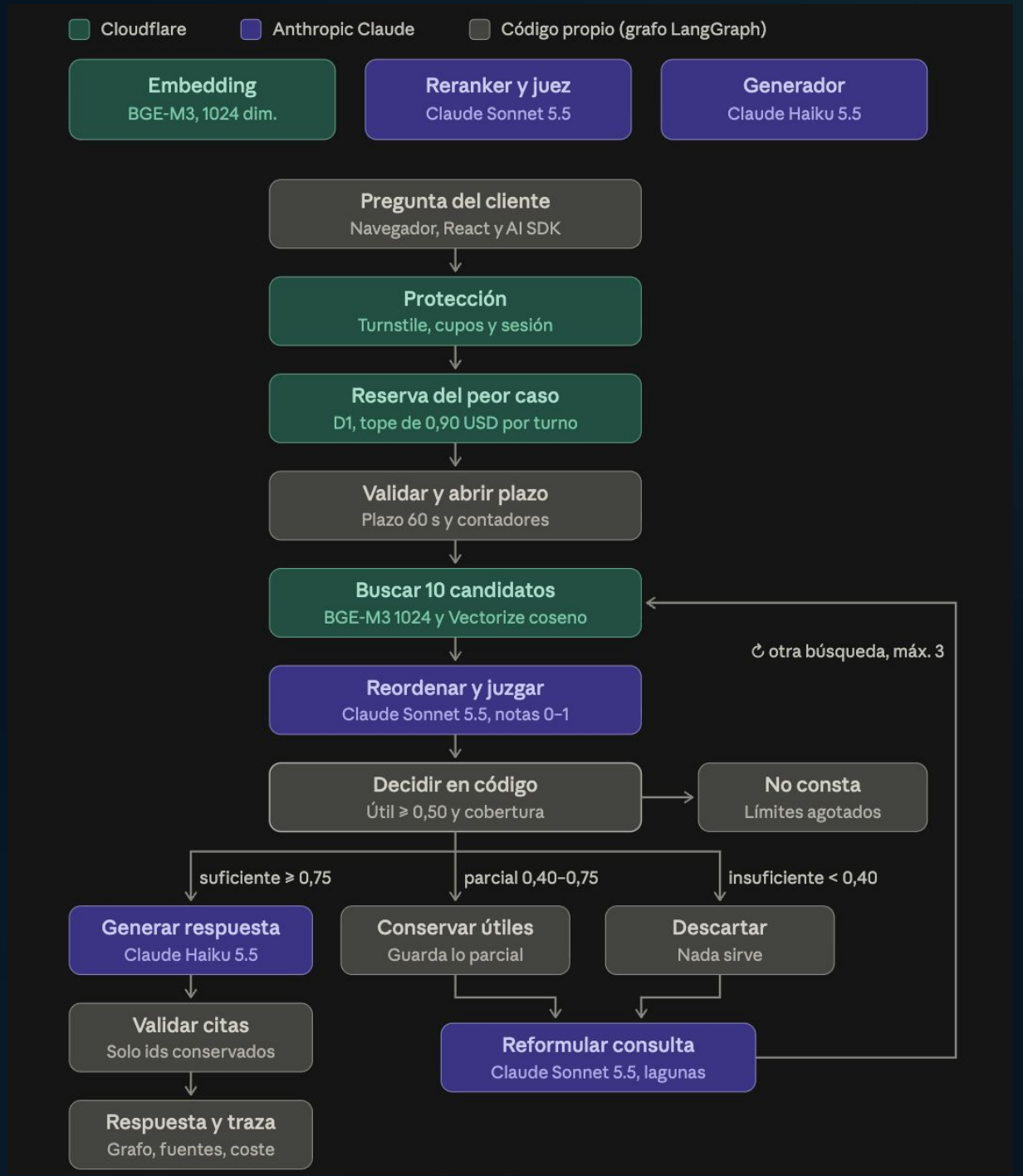
Casos 3 y 8. Cobertura parcial tras 2 reformulaciones: CRAG rechaza por prudencia.

lab/ · 10 documentos AetherCloud, vector(768) + HNSW, Qwen3-Reranker-0.6B (lab/eval/resultados-laboratorio.md). Lote histórico del asistente (3 oct): 17/20 → 18/20 y latencia mediana 1,19 s → 4,30 s. Muestra pequeña: no es una tasa general.

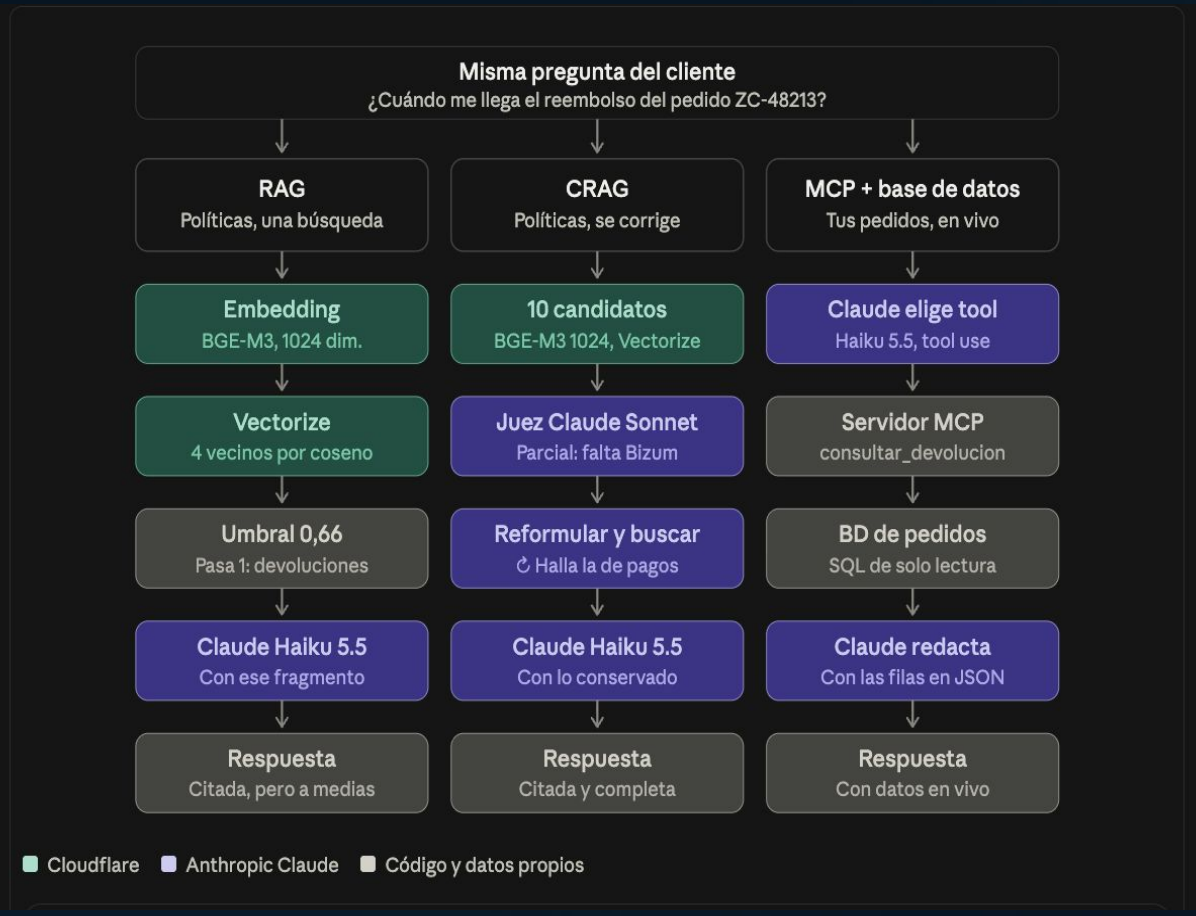
El RAG de nuestro monorepo



El CRAG de nuestro monorepo



comparamos las tecnicas



«Devolví el pedido ZC-48213 en un Zocopunto y lo pagué con Bizum. ¿Cuándo me llega el dinero?»

RAG

Una búsqueda · Haiku 5.5

> «El almacén tiene hasta 3 días hábiles desde que recibe el paquete para validar la devolución; después ordena el reembolso por el mismo método de pago [02-devoluciones]. No consta cuánto tarda el abono con Bizum ni el estado de tu pedido.»

- ⚠ Plazo a medias: Bizum está en otro documento
- ✗ No sabe en qué punto está tu pedido
- ✓ Cita lo que usa y no inventa
- ⚡ 1 llamada a Haiku · reserva 0,0075 USD

CRAG

Juez y reformulación Sonnet · Haiku 5.5

> «El almacén tiene hasta 3 días hábiles desde que recibe el paquete para validar la devolución [02-devoluciones] y Bizum abona en 2 días hábiles una vez ordenado el reembolso [06-pagos]. En total, hasta 5 días hábiles. No consta el estado de tu pedido ZC-48213.»

- ✓ Plazo completo: junta los 2 documentos
- ✗ No sabe en qué punto está tu pedido
- ✓ Cita los dos documentos
- ⏱ 2 búsquedas y 2 llamadas Sonnet · reserva 0,90 USD

MCP + base de datos

Tabla de devoluciones · Haiku 5.5

> «La devolución de tu pedido ZC-48213 se validó y el reembolso por Bizum se ordenó el viernes 23/10. Todavía no figura como abonado. La base de datos no indica cuánto tarda Bizum.»

- ✗ No conoce el plazo de la política
- ✓ Estado exacto y en vivo de tu pedido
- ✓ Trazable a filas de la BD
- 🔒 Exige identificar al cliente